CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY
Faculty of Technology & Engineering
Devang Patel Institute of Advance Technology and Research (DEPSTAR)
Data Engineering (ITUC301)
Practical File – ODD Semester (2026-27)

Student ID:24DIT043
Student Name : Krushil Patel

Practical 2

Problem Definition

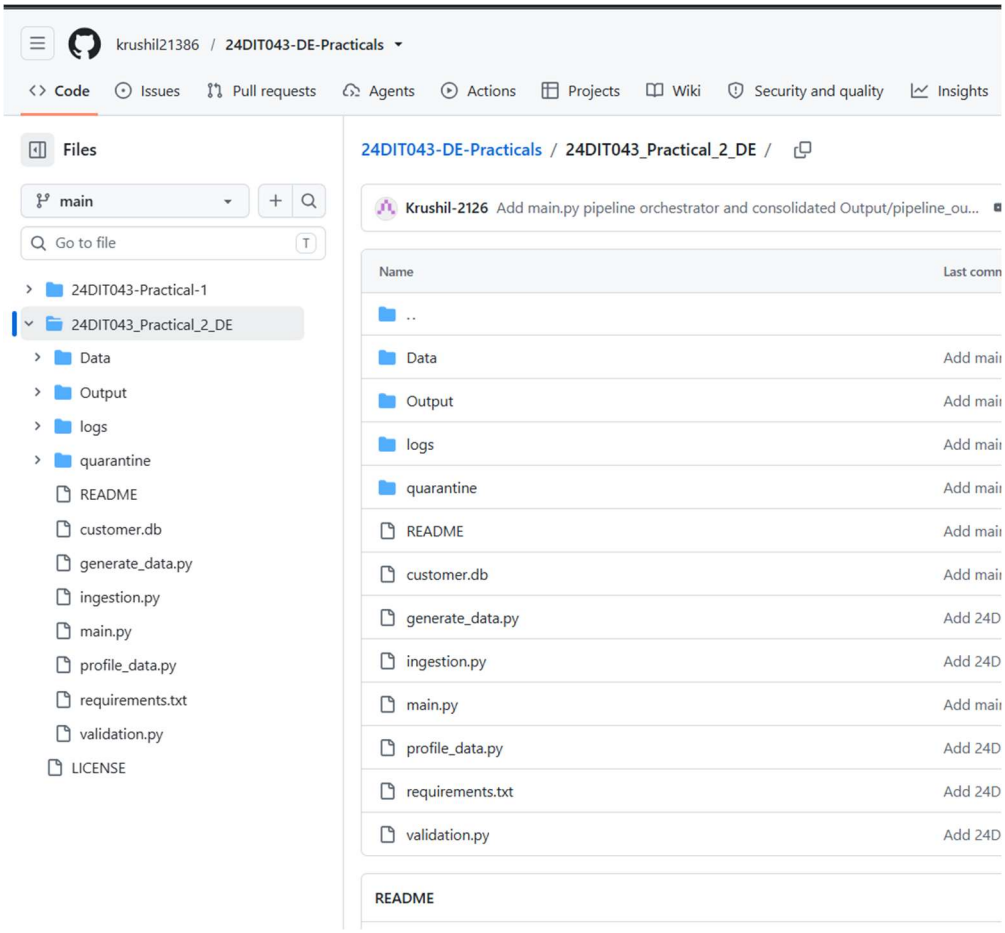
Programmatically interface with diverse file-based, API-driven, and relational source database environments. Students will write generation engines to simulate transactional workloads, profile the incoming files to automatically discover schemas, isolate data quality violations (such as null entry leaks and format divergence), and establish appropriate operational ingestion criteria.

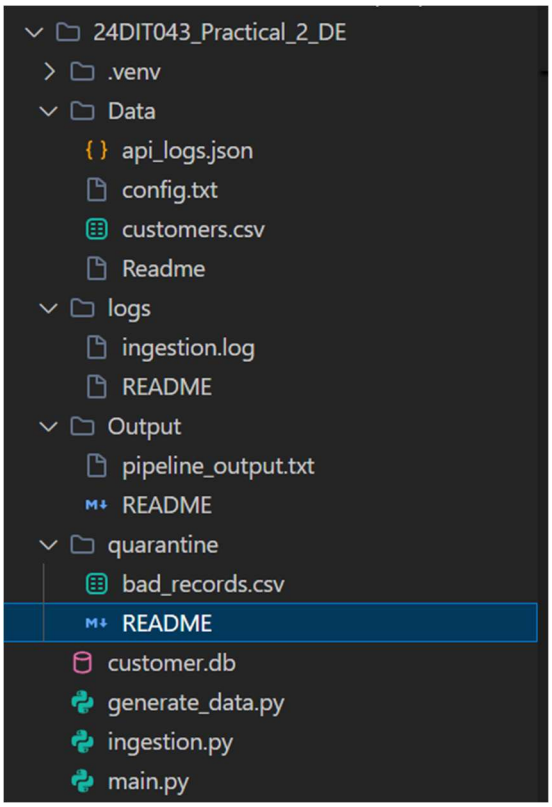
Dataset / Test DataData: Programmatically simulated customer profile tables, semi-structured API transaction logs containing nested objects, and unstructured configuration text files..

Tools / Technology UsedPython 3.x, Faker library, Pandas, Requests HTTP library, SQLite/PostgreSQL, and Git..

Total Hours of Problem Definition Implementation: 2 Hours

OUTPUTS:




24DIT043_Practical_2_DE
.venv
Data
api_logs.json
config.txt
customers.csv
Readme
logs
ingestion.log
README
Output
pipeline_output.txt
README
quarantine
bad_records.csv
README
customer.db
generate_data.py
ingestion.py
main.py

24DIT043_Practical_2_DE > Data > {} api_logs.json > ...


```json
[
    {
        "id": 1,
        "customer": {
            "name": "Erica Harris",
            "city": "West Michael"
        },
        "order": {
            "amount": 3564
        }
    },
    {
        "id": 2,
        "customer": {
            "name": "Marcus Edwards",
            "city": "West Raymond"
        },
        "order": {
            "amount": 3869
        }
    },
    {
        "id": 3,
        "customer": {
            "name": "Miss Christie Bruce MD",
            "city": "New Jeffreyside"
        },
        "order": {
            "amount": 3779
        }
    },
    {
        "id": 4,
        "customer": {
            "name": "Bill Green"
        },
        "order": {
            "amount": 1089
        }
    },
    {
        "id": 5,
        "customer": {
            "name": "Holly Scott",
            "city": "North Brendan"
        },
        "order": {
            "amount": 1422
        }
    },
    {
        "id": 6,
        "customer": {
```

```
(.venv) (base) PS C:\Users\5430\Downloads\Data-Engineering-Practicals-main\Data-Engineering-Practicals-main\24DIT043_Practical_2_DE> python quality_check.py
>>

==================================================
DATA QUALITY REPORT
==================================================

Total Records          : 100
Missing Customer IDs   : 5
Invalid Phone Numbers  : 4

Invalid Phone Numbers

    customer_id   full_name         email                     phone_number   signup_date  credit_score
0   CUST-83810    Aryan Maharaj     udantdewan@hotmail.com    INVALID_9999   2024-10-17   581
3   CUST-72357    Janaki Handa      bathvedant@yahoo.com      INVALID_9999   2024-09-29   395
22  CUST-66768    Garima Bala       lalpranav@hotmail.com     INVALID_9999   2025-02-21   623
80  CUST-50667    Zayyan Doctor     finn92@yahoo.com          INVALID_9999   2025-03-27   383

Missing Customer IDs

    customer_id   full_name           email                     phone_number    signup_date  credit_score
36  NaN           Amrita Naidu        gabrielmallick@yahoo.com    07262849877     2025-02-04   569
42  NaN           Upkaar Virk         cseshadri@yahoo.com         +919022941318   2026-05-29   614
53  NaN           Edhitha Sura        biswaswahab@hotmail.com     +916217459615   2026-01-12   360
85  NaN           Chatresh Randhawa   warinder61@yahoo.com        +914782613750   2025-01-20   402
91  NaN           Widisha Pillai      fitan61@yahoo.com          +915415667652   2025-11-09   606

Accepted Records : 91
Rejected Records : 9
Quarantine file created.

Database Preview

    customer_id   full_name        email                    phone_number   signup_date  credit_score
0   CUST-88696    Rushil Saini     saumyamall1@gmail.com    8196001338     2026-03-11   404
1   CUST-20926    Hemangini Lalla  sharafjeet@yahoo.com     6542351161     2024-09-29   389
2   CUST-30512    Yash Gade        rraghavan@hotmail.com    +916475255341  2026-01-15   327
4   CUST-32325    Gavin Batta      duanathaniel@hotmail.com 8849696532     2025-06-22   306
```

```python
accepted = df[
    (df["customer_id"].notnull()) &
    (df["phone_number"] != "INVALID_9999")
]

rejected = df[
    (df["customer_id"].isnull()) |
    (df["phone_number"] == "INVALID_9999")
]

print("\nAccepted Records :", len(accepted))
print("Rejected Records :", len(rejected))
```

```python
# ----------------------------
# Ingestion Validation
# ----------------------------
accepted = df[
    (df["customer_id"].notnull()) &
    (df["phone_number"] != "INVALID_9999")
]

rejected = df[
    (df["customer_id"].isnull()) |
    (df["phone_number"] == "INVALID_9999")
]

print("\nAccepted Records :", len(accepted))
print("Rejected Records :", len(rejected))
```

24DIT043_Practical_2_DE > Data > customers.csv

```
CustomerID,Name,Email,City
1.0,Lindsey Fisher,racheljackson@example.net,Derekborough
2.0,Brittany Rivera,heidijones@example.net,Port Joshua
3.0,Sean Fowler,,East Francisco
4.0,John York,lewisvalerie@example.net,Andrewport
5.0,Nancy Novak,sdavis@example.com,Wallerland
6.0,Harold Crawford,bowerskristina@example.org,
7.0,Brittany Alvarez,hayestaylor@example.net,Lake Michaelville
8.0,Miguel Munoz,macdonalddoris@example.org,Simmonshaven
,Russell Long,hendersonkatelyn@example.net,Nathanielland
10.0,Cheryl Jones,josegarcia@example.org,North Emilyberg
11.0,Sharon Conway,mikeromero@example.org,South Tiffanybury
12.0,Jennifer Mcbride,jhunt@example.com,Lake Shannon
13.0,Pamela Dalton,jason01@example.com,New Douglas
14.0,Michelle Holder,thompsonelizabeth@example.org,Brittanyfurt
15.0,James Mahoney,ann08@example.org,South Davidport
16.0,Elizabeth Mcintosh,darlene90@example.com,North David
17.0,Jessica Jimenez,cleach@example.org,Blanchardville
18.0,Kayla Morris,joyce26@example.com,South Tanyaton
19.0,Douglas Gibbs,tyler29@example.net,New Kariton
20.0,Cassandra Hall,ihill@example.com,East Gregoryland
Ctrl+L for Agent
```

README ...\Output | README ...\quarantine | quality_check.py | *bad_records.csv* | main.py

24DIT043_Practical_2_DE > quarantine > bad_records.csv

```
1  CustomerID,Name,Email,City
2  3.0,Sean Fowler,,East Francisco
3  6.0,Harold Crawford,bowerskristina@example.org,
4  ,Russell Long,hendersonkatelyn@example.net,Nathanielland
5
```

KEY QUESTIONS

1. Explain the architectural strategies used to manage severe structural schema drift when handling continuous external third-party API inputs.

Schema drift occurs when the structure of incoming data changes over time, such as adding new fields, removing fields, or changing data types in third-party API responses. To handle schema drift, modern data architectures use flexible ingestion strategies like **schema-on-read**, where raw data is stored first and interpreted later during processing.

Some common strategies include:

- **Raw Data Landing Zone:** Store incoming API data in its original format without modifying it.
- **Schema Validation Layer:** Check incoming data against predefined rules before processing.
- **Data Versioning:** Maintain different schema versions to support changes from external systems.
- **Metadata Management:** Track changes in fields, data types, and structures automatically.
- **ETL/ELT Pipelines:** Transform data into a consistent format before loading into the target database.

These approaches allow systems to handle continuous API changes without breaking the entire data pipeline.

2. What are the structural benefits and data storage trade-offs of using semi-structured nested JSON objects over heavily normalized SQL database records?

Semi-structured JSON data provides flexibility because it allows storing changing and complex data structures without modifying the database schema frequently.

Benefits of Nested JSON:

- Handles changing schemas easily.
- Stores hierarchical data naturally.
- Reduces the need for multiple table joins.
- Faster ingestion of API responses.
- Suitable for unstructured and semi-structured data sources.

Trade-offs:

- Requires more storage space due to repeated fields.
- Data duplication may occur.
- Complex queries can be slower compared to normalized SQL tables.
- Data validation becomes more challenging.
- Maintaining relationships between records is difficult.

Therefore, JSON is preferred for flexible ingestion, while normalized SQL databases are preferred for structured reporting and transactions.

3. How do you validate data quality at the point of ingestion without creating severe processing performance bottlenecks?

Data quality validation can be performed efficiently by applying lightweight checks during the ingestion stage instead of performing expensive transformations immediately.

Techniques include:

- **Schema validation:** Check whether required fields and data types are correct.
- **Primary key validation:** Ensure unique identifiers exist.
- **Range validation:** Verify values are within acceptable limits.
- **Null checks:** Detect missing mandatory fields.
- **Sampling:** Validate a percentage of records instead of every record when datasets are extremely large.
- **Parallel processing:** Perform validation using distributed processing frameworks.

Invalid records should be redirected to a quarantine area while valid records continue through the pipeline.

Supplementary Problem: Validation Routine for Messy User Files

Objective:

Develop a validation system that reads multiple user files, detects invalid records, moves them to quarantine storage, and generates warning logs.

Validation Process:

1. Read files from the input directory.
2. Parse each record.
3. Check mandatory fields (primary keys).
4. Validate data types and formats.
5. Separate valid and invalid records.
6. Store invalid records in quarantine.
7. Generate system logs and alerts.

Example Validation Rules:

Validation Check	Action
Missing User ID	Move record to quarantine
Invalid email format	Generate warning
Empty mandatory fields	Reject record
Duplicate primary key	Flag duplicate

Key Skills Covered

1. Programmatic Data Profiling

Analyzing incoming data to understand:

- Data types
- Missing values
- Duplicate records
- Data distribution

2. Synthetic Record Generation

Creating sample data to test validation rules and system performance**.**

3. Automated Quality Boundary Enforcement

Applying rules automatically to accept, reject, or quarantine records.

4. Source System Type Evaluation

Understanding whether data comes from:

- APIs
- Databases
- CSV files
- JSON files
- IoT sources

Applications

1. Source Data Profiling Layer

Analyzes incoming data quality before processing.

2. Raw Data Landing Zone

Stores original incoming data safely.

3. Automated Incoming Payload Quality Gates

Blocks poor-quality data from entering production systems**.**

Conclusion:

A robust data ingestion system requires automated profiling, schema validation, quality checks, and quarantine mechanisms. These techniques ensure reliable, accurate, and consistent data enters the target system even when external sources continuously change.